

PROJECT ETHER

Blueprint v3.0

Especificação de Domínio, Dados, Protocolos, Fórmulas, Testes e Implementação

Status: Implementation Specification

Baseline: Blueprint v2.0

Objetivo: permitir implementação incremental por equipe humana ou agente de desenvolvimento sem inventar regras de negócio silenciosamente.

1. REGRA DESTE DOCUMENTO

Este documento é a fonte de verdade do sistema.

Hierarquia:

```
Product Decision
  ↓
Game Design Rule
  ↓
Domain Model
  ↓
Application Contract
  ↓
Infrastructure
  ↓
Client Presentation
```

Código não pode redefinir uma regra de produto.

Quando houver conflito:

SPEC_CONFLICT

Quando faltar definição:

SPEC_GAP

Quando depender de decisão de produto:

PRODUCT_DECISION_REQUIRED

2. OBJETIVO DO MVP TÉCNICO

O primeiro sistema jogável deve suportar:

Account
↓
Character
↓
World
↓
Movement
↓
Creature
↓
Combat
↓
Loot
↓
Inventory
↓
Equipment
↓
Experience
↓
Level
↓
Quest
↓
Death
↓
Respawn

↓
Logout
↓
Login
↓
State restored

E simultaneamente:

Player A ↔ Server ↔ Player B

3. SOLUTION

```
src/  
  Ether.Domain/  
  Ether.Application/  
  Ether.Infrastructure/  
  Ether.Contracts/  
  Ether.Api/  
  Ether.GameServer/  
  Ether.Worker/  
  
tests/  
  Ether.Domain.Tests/  
  Ether.Application.Tests/  
  Ether.Infrastructure.Tests/  
  Ether.Api.Tests/  
  Ether.GameServer.Tests/  
  Ether.Architecture.Tests/  
  
client/  
  Ether.Game/  
  
tools/  
  Ether.ContentTools/  
  Ether.AdminTools/  
  
docs/  
  architecture/
```

game-design/
protocol/
balance/
economy/
operations/

4. DEPENDENCY RULE

Dependências permitidas:

Domain

↓

Application

↓

Infrastructure

Contracts

↑

Application / API / GameServer / Client

Domain não pode depender de:

- EF Core;
- PostgreSQL;
- Redis;
- HTTP;
- Godot;
- WebSocket;
- ASP.NET.

5. DOMAIN MODEL

5.1 Aggregate Roots

Principais aggregates:

Account
Character
Inventory
Wallet
MarketListing
Trade
QuestState
Guild
Party
CraftingJob

World entities de alta frequência não devem ser tratados como aggregates persistidos a cada tick.

6. VALUE OBJECTS

Criar value objects para:

CharacterId
AccountId
ItemInstanceId
ItemDefinitionId
MapId
CreatureId
QuestId
GuildId
Position
Money
Experience
Health
Mana
Level
SkillLevel
Duration

Evitar espalhar Guid, int e decimal indiscriminadamente.

7. IDENTIFIERS

Todos os IDs de domínio devem ser não previsíveis externamente.

Preferência:

Guid / UUID

ou identificador equivalente.

IDs internos sequenciais podem existir onde houver necessidade de performance, mas não devem ser expostos como mecanismo de autorização.

8. MONEY

Nunca utilizar `double` ou `float` para dinheiro.

Modelo:

```
Money
{
    Currency
    Amount
}
```

`Amount` deve possuir precisão determinística.

Para Gold:

`non-negative integer`

Para moedas fracionárias futuras:

`decimal`

9. CHARACTER

Character

```
{  
    CharacterId  
    AccountId  
    Name  
    ClassId  
    SpecializationId?  
    Level  
    Experience  
    Attributes  
    Health  
    Mana  
    Position  
    State  
}
```

Invariantes:

```
Level >= 1  
Experience >= 0  
Health >= 0  
Mana >= 0  
Name unique  
Account ownership valid  
Position valid for current map
```

10. CHARACTER STATE

Offline
Connecting
Loading
InWorld
Combat
DisconnectGrace
Dead

Respawning

Transições inválidas devem ser rejeitadas.

11. CHARACTER ATTRIBUTES

Strength
Dexterity
Intelligence
Vitality
Willpower

Base values são derivados de:

Class
Level
Allocation
Equipment
Buffs
Debuffs

Não gravar valores derivados como autoridade quando podem ser recalculados.

12. STAT PIPELINE

Sempre:

Base
↓
Level
↓
Class
↓
Allocation
↓

Equipment
↓
Permanent modifiers
↓
Temporary buffs
↓
Temporary debuffs
↓
Final Stat

O sistema deve permitir inspecionar cada etapa.

Isso será fundamental para debugging e balanceamento.

13. CLASS DEFINITION

```
ClassDefinition
{
    Id
    Name
    BaseAttributes
    GrowthRules
    AllowedWeapons
    StartingAbilities
}
```

Classes iniciais:

Warrior
Ranger
Arcanist

14. ABILITY

```
AbilityDefinition
{
    Id
    Name
    Type
    ResourceType
    ResourceCost
    Cooldown
    Range
    Area
    TargetRule
    DamageProfile
    StatusEffects
    Animation
}
```

Tipos:

```
BasicAttack
Active
Passive
Utility
Defensive
Movement
```

15. COMMAND MODEL

Todo comando possui:

```
CommandId
SessionId
CharacterId
ClientSequence
Timestamp
Payload
```

Exemplo:

```
MoveCommand
AttackCommand
CastAbilityCommand
InteractCommand
PickupCommand
EquipCommand
```

16. COMMAND IDEMPOTENCY

CommandId deve ser único.

Se o servidor receber novamente:

CommandId = X

não executa novamente.

Pode devolver o resultado anteriormente registrado quando aplicável.

17. MOVEMENT

Input:

```
MoveCommand
{
    Destination
}
```

ou:

```
MoveDirectionCommand
{
    Direction
}
```

Servidor valida:

Map
Tile
Collision
Speed
CurrentState
Zone
Teleport restrictions

18. MOVEMENT RULE

O cliente pode dizer:

“quero ir para X”.

Não pode dizer:

“estou em X”.

O servidor calcula a posição autorizada.

19. POSITION

Position

```
{  
    MapId  
    X  
    Y  
}
```

Se futuramente houver verticalidade:

Z

será introduzido no contrato.

20. MOVEMENT SPEED

Velocidade final:

```
BaseMovementSpeed
× EquipmentModifier
× BuffModifier
× DebuffModifier
× TerrainModifier
```

Sempre aplicar limites:

```
MinimumSpeed
MaximumSpeed
```

21. CREATURE INSTANCE

```
CreatureInstance
{
    InstanceId
    DefinitionId
    MapId
    Position
    Health
    State
    TargetId?
    SpawnId
}
```

Estado:

```
Idle
Patrol
Investigate
Chase
```

Attack
Flee
Return
Dead
Respawning

22. CREATURE SPAWN

Spawn configuration:

```
SpawnDefinition
{
    SpawnId
    MapId
    Position
    CreatureDefinitionId
    MaxAlive
    RespawnTime
    Radius
}
```

Respawn deve ser controlado pelo servidor.

23. COMBAT COMMAND

```
AttackCommand
{
    CommandId
    TargetId
}
```

Servidor verifica:

1. character existe;
2. character está vivo;

3. character pode atacar;
4. target existe;
5. target é válido;
6. target está vivo;
7. range;
8. line of sight;
9. cooldown;
10. weapon;
11. resource;
12. combat state.

Somente então calcula dano.

24. DAMAGE PIPELINE

Base Damage



Weapon Power



Attribute Scaling



Skill Scaling



Ability Modifier



Critical



Resistance



Armor



Flat Modifiers



Minimum Damage Rule



Final Damage

25. DAMAGE FORMULA V1

RawDamage =
AbilityBase
+ WeaponPower
+ ScalingFromAttributes
+ ScalingFromSkill

Armor:

ArmorMultiplier =
 $100 / (100 + \text{EffectiveArmor})$

Resistance:

ResistanceMultiplier =
 $1 - \text{Clamp}(\text{EffectiveResistance}, 0, \text{ResistanceCap})$

Final:

FinalDamage =
max(
 MinimumDamage,
 RawDamage
 × ArmorMultiplier
 × ResistanceMultiplier
 × CriticalMultiplier
)

26. CRITICAL

CriticalChance
CriticalMultiplier

Initial:

`CriticalMultiplier = 1.5`

`PRODUCT_BALANCE_REQUIRED` para chance e caps definitivos.

Randomização ocorre exclusivamente no servidor.

27. MINIMUM DAMAGE

Para evitar criaturas completamente imunes:

`MinimumDamage = 1`

Exceções podem existir para habilidades específicas.

28. HEALING

Pipeline:

```
BaseHealing
+ AbilityScaling
+ AttributeScaling
+ EquipmentModifiers
+ Buffs
```

Depois:

```
FinalHealing =
clamp(
    CalculatedHealing,
    0,
    MaxMissingHealth
)
```

29. STATUS EFFECTS

Generic:

```
StatusEffect
{
    Id
    Type
    Source
    Target
    StartedAt
    ExpiresAt
    Magnitude
    Stacks
}
```

Stacking policy deve ser explícita:

```
RefreshDuration
AddStacks
ReplaceIfStronger
Reject
```

Nunca implicitamente acumular.

30. TARGETING

Target rules:

```
Self
SingleEnemy
SingleAlly
Area
Cone
Line
Ground
Projectile
```

Cada habilidade declara sua regra.

31. LINE OF SIGHT

LOS é calculado server-side.

Tiles com:

`BlocksSight = true`

interrompem habilidades que exigem LOS.

32. AREA OF EFFECT

Server calcula:

`Origin`

`Shape`

`Radius`

`AffectedEntities`

Não aceitar lista de alvos fornecida pelo cliente.

O cliente apenas solicita:

`CastAbility`

`AbilityId`

`TargetPosition`

33. EXPERIENCE

Eventos que podem gerar XP:

- creature kill;

- quest;
- exploration;
- discovery;
- crafting;
- event.

Cada fonte deve declarar:

ExperienceSource

34. XP GRANT

```
GrantExperience(  
    CharacterId,  
    Amount,  
    Source,  
    ReferenceId  
)
```

A operação é idempotente quando associada a um evento único.

35. LEVEL UP

Ao cruzar o threshold:

```
Experience  
↓  
LevelCalculation  
↓  
LevelChanged  
↓  
AttributeUpdate  
↓  
AbilityUnlock  
↓
```

Reward

Level nunca deve ser incrementado diretamente pelo cliente.

36. LEVEL CURVE V1

Tabela inicial recomendada para prototipagem:

```
XPRequired(level) =  
100 × level^1.65
```

Arredondamento:

```
floor()
```

Essa fórmula é provisória.

Balanceamento final será determinado por playtest.

37. SKILL

```
Skill  
{  
    CharacterId  
    SkillId  
    Level  
    Experience  
}
```

Skills não precisam compartilhar exatamente a mesma curva do personagem.

38. INVENTORY

```
Inventory
{
    CharacterId
    Slots[]
}
```

Slot:

```
InventorySlot
{
    Index
    ItemInstanceId?
    Quantity
}
```

39. ITEM OWNERSHIP

Todo item possui exatamente uma localização lógica:

```
Inventory
Equipment
Market
Trade
Crafting
World
Destroyed
```

Nunca duas simultaneamente.

40. ITEM TRANSFER

Toda transferência:

Source
↓
Validation
↓
Lock
↓
Ownership Change
↓
Audit
↓
Commit

41. ITEM CREATION

Item só pode nascer através de um `ItemCreationSource`.

Exemplos:

Loot
Crafting
QuestReward
Purchase
AdminGrant
EventReward

Cada criação possui:

`CreationReferenceId`

42. ITEM DESTRUCTION

Item só pode ser destruído através de:

`DestroyItemCommand`

com:

Reason

Source

ReferenceId

43. EQUIPMENT

Equipar:

Validate item

Validate slot

Validate class

Validate level

Validate requirements

Remove from inventory

Place in equipment

Commit

Se qualquer etapa falhar:

No state change

44. QUEST ENGINE

Quest:

QuestDefinition

{

Id

Name

Requirements

Objectives

Rewards

Expiration?

```
}
```

State:

```
QuestState  
{  
    CharacterId  
    QuestId  
    Status  
    ObjectiveProgress  
}
```

45. OBJECTIVE TYPES

MVP:

```
KillCreature  
CollectItem  
TalkToNpc  
ReachLocation  
UseAbility
```

Futuro:

```
DiscoverLocation  
CraftItem  
Escort  
Puzzle  
ChooseBranch
```

46. QUEST REWARDS

Possible:

Experience
Gold
Item
SkillExperience
Reputation
Unlock
Cosmetic

Rewards devem ser entregues atomicamente.

47. NPC

```
NpcDefinition
{
    Id
    Name
    MapId
    Position
    DialogueId
    ShopId?
    QuestProviderId?
    FactionId?
}
```

48. DIALOGUE

Dialogue deve ser data-driven.

```
Dialogue
  Node
    ↓
  Option
    ↓
  Node
```

Options podem possuir:

Requirement

Action

ReputationRequirement

QuestRequirement

49. WALLET

Wallet

```
{  
    CharacterId  
    Gold  
    PremiumCurrency  
}
```

Mas o saldo não é a única fonte de verdade.

Também existe:

CurrencyTransaction

50. CURRENCY TRANSACTION

CurrencyTransaction

```
{  
    Id  
    CharacterId  
    Currency  
    Amount  
    Direction  
    BalanceBefore  
    BalanceAfter  
    Source  
    ReferenceId  
    CreatedAt
```

```
}
```

51. WALLET INVARIANTS

Nunca:

`Balance < 0`

exceto moedas explicitamente configuradas como dívida, o que inicialmente não existe.

Toda movimentação possui referência.

52. MARKETPLACE

MarketListing

```
{  
    Id  
    SellerId  
    ItemId  
    Quantity  
    UnitPrice  
    Currency  
    Status  
    CreatedAt  
    ExpiresAt  
}
```

Quando listado:

Item

↓

Inventory

↓

MarketEscrow

O vendedor não pode usar o item enquanto estiver listado.

53. MARKET PURCHASE

Transaction:

BEGIN

Lock Listing
Lock Buyer Wallet
Validate Listing Active
Validate Quantity
Validate Balance
Debit Buyer
Transfer Item
Apply Tax
Credit Seller
Mark Listing Sold/Partial
Write Audit
Commit

Essa operação deve ser atômica.

PostgreSQL constraints devem reforçar invariantes onde apropriado; constraints existem justamente para impedir que dados inválidos sejam persistidos.

54. MARKET TAX

Tax é um economic sink.

Modelo:

Tax =
SaleValue × MarketTaxRate

Taxa inicial:

PRODUCT_BALANCE_REQUIRED

55. TRADE

Trade possui dois lados:

TradeParticipant A

TradeParticipant B

Cada lado possui:

Items

Currency

Confirmed

56. TRADE INVALIDATION

Qualquer alteração em:

- item;
- quantidade;
- gold;

zera:

Confirmed = false

para ambos.

57. TRADE EXECUTION

Somente:

```
A.Confirmed == true  
AND  
B.Confirmed == true
```

permite execução.

58. CRAFTING

```
CraftingRecipe  
{  
    Id  
    ResultItem  
    Ingredients  
    RequiredProfession  
    RequiredLevel  
    Station?  
    Duration  
}
```

59. CRAFTING JOB

```
CraftingJob  
{  
    Id  
    CharacterId  
    RecipeId  
    StartedAt  
    CompletesAt  
    Status  
    InputSnapshot  
}
```

60. CRAFTING INPUT

Ingredientes são removidos quando o job inicia.

Se o job for cancelável:

CancellationPolicy

deve definir exatamente o que é devolvido.

61. QUALITY

Crafting pode calcular:

QualityScore

A qualidade é influenciada por:

- profession level;
- recipe;
- material quality;
- optional modifiers;
- random roll.

Random server-side.

62. RANDOMNESS

Todo RNG crítico deve ser:

- server-side;
- auditável;
- associado a uma operação.

Não enviar seed manipulável pelo cliente.

63. LOOT RNG

Ao morrer:

```
LootRollContext
{
    CreatureId
    KillerId
    PartyContext
    LootTable
    WorldModifiers
}
```

Servidor calcula.

64. PARTY

Party aggregate:

```
Party
{
    PartyId
    LeaderId
    Members[]
    LootPolicy
    ExperiencePolicy
}
```

65. PARTY XP

MVP:

XP é distribuído entre membros elegíveis.

Eligibility pode considerar:

- mesma área;
- participação;
- level range.

A fórmula final será balanceada.

66. GUILD

Guild:

```
Guild
{
    Id
    Name
    LeaderId
    Level
    Experience
}
```

Membership:

```
GuildMember
{
    GuildId
    CharacterId
    Rank
    JoinedAt
}
```

Unique:

CharacterId

Um personagem não pode pertencer a duas guilds simultaneamente no MVP.

67. CHAT

Canais:

Local
Party
Guild
Private
World

World chat pode possuir rate limit.

68. ANTI-SPAM

Por canal:

MinimumMessageInterval
MaximumLength
FloodWindow
MuteDuration

69. FRIENDS

Friend relation:

Pending
Accepted
Blocked

Não permitir messaging quando bloqueado.

70. REPORT

Report:

ReportId
Reporter
Target
Category
Description
Evidence
CreatedAt
Status

Status:

Open
Investigating
Resolved
Rejected

71. NETWORK PROTOCOL

Endpoint:

`wss://api.<domain>/game`

Godot possui suporte a WebSocket através de `WebSocketPeer`, inclusive em exports mobile; no Android, a permissão de Internet precisa estar habilitada.

ASP.NET Core também suporta WebSockets diretamente e possui mecanismos para keep-alive e detecção de desconexão.

72. FRAME

Cada mensagem possui envelope:

```
{  
  "version": 1,  
  "type": "MoveCommand",  
  "requestId": "uuid",  
  "sequence": 123,  
  "payload": {}  
}
```

73. SERVER MESSAGE

```
{  
  "version": 1,  
  "type": "EntityMoved",  
  "sequence": 124,  
  "payload": {}  
}
```

74. ERROR MESSAGE

```
{  
  "version": 1,  
  "type": "CommandRejected",  
  "requestId": "uuid",  
  "code": "OUT_OF_RANGE",  
  "message": "Target is out of range."  
}
```

Mensagens de erro não devem revelar informações sensíveis.

75. ERROR CODES

Exemplos:

INVALID_COMMAND
INVALID_STATE
NOT_AUTHORIZED
NOT_FOUND
OUT_OF_RANGE
LINE_OF_SIGHT_BLOCKED
COOLDOWN_ACTIVE
INSUFFICIENT_RESOURCE
INVALID_TARGET
INVENTORY_FULL
INSUFFICIENT_FUNDS
ITEM_NOT_OWNED
TRADE_INVALIDATED
LISTING_UNAVAILABLE

76. SEQUENCE

Cada cliente possui sequência monotônica.

Servidor pode detectar:

duplicate
replay
out-of-order

Comandos idempotentes podem ser repetidos com segurança.

77. HEARTBEAT

WebSocket deve possuir heartbeat.

O servidor deve detectar:

Connected
Stale
Disconnected

Nunca confiar exclusivamente no evento de socket close.

78. RECONNECT

Client:

ConnectionLost



Backoff



Reconnect



Authenticate



ResumeSession



StateSnapshot



DeltaSync



InWorld

79. STATE SNAPSHOT

Snapshot deve incluir somente o necessário:

Character

Position

Health

Mana

Equipment

Inventory

ActiveQuests

NearbyEntities

Cooldowns

ActiveEffects

80. ENTITY REPLICATION

Servidor envia:

Spawn

Update

Despawn

Não enviar todos os atributos internos.

Somente estado necessário para o cliente.

81. INTEREST MANAGEMENT

Cada sessão possui:

InterestArea

Entidades fora da área não precisam ser transmitidas.

82. TICK MODEL

Inicial:

Movement: 10-20 Hz

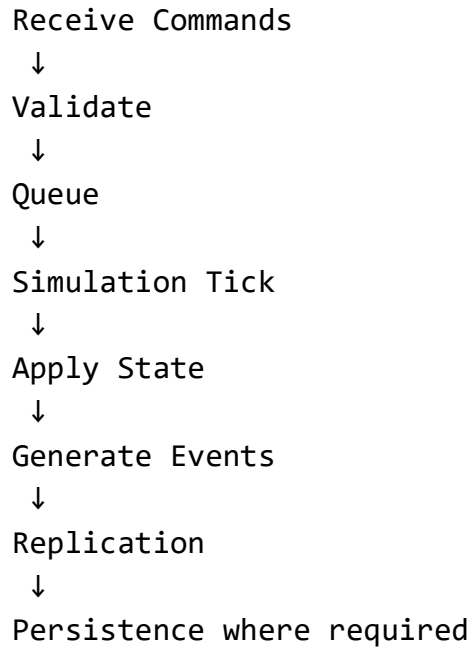
Combat: 10-20 Hz

AI: 5-10 Hz

O valor definitivo deve ser determinado por profiling.

O objetivo é não executar a mesma frequência para todos os subsistemas.

83. WORLD LOOP



84. PERSISTENCE STRATEGY

Não salvar cada posição em PostgreSQL a cada tick.

Persistir:

- eventos importantes;
- checkpoints;
- logout;
- disconnect grace;
- mudança de mapa;
- alterações econômicas;
- alterações de inventário;
- progressão;
- quest state.

85. WORLD STATE

World state de alta frequência fica em memória do Game Server.

PostgreSQL é fonte de persistência.

Redis pode auxiliar em:

- session coordination;
- distributed locks;
- transient state;
- queues;
- pub/sub.

Redis não deve virar fonte permanente de verdade do personagem.

86. TRANSACTION BOUNDARIES

Operações críticas:

Loot

Trade

Market

Wallet

Crafting

QuestReward

Purchase

ItemTransfer

devem definir explicitamente sua transaction boundary.

87. DATABASE SCHEMA — CORE

accounts

characters

character_attributes

character_skills

character_equipment
inventory_slots

item_definitions
item_instances

maps
map_tiles
spawn_definitions
creature_definitions

quest_definitions
quest_states

npc_definitions
dialogue_definitions

wallets
currency_transactions

market_listings
trades
trade_items

crafting_recipes
crafting_jobs

guilds
guild_members
parties
party_members

friends
blocks
reports

notifications
audit_events

88. PRIMARY KEYS

Todos os aggregates utilizam:

UUID

Foreign keys obrigatórias devem possuir índices quando necessários para os padrões de consulta.

89. DATABASE CONSTRAINTS

Exemplos:

```
CHECK (amount >= 0)
CHECK (quantity > 0)
CHECK (level >= 1)
CHECK (health >= 0)
```

Unique:

```
UNIQUE(account_id, name)
```

ou regras equivalentes.

Foreign keys devem ser usadas para invariantes relacionais reais.

90. ITEM INSTANCE CONSTRAINT

Um `ItemInstance` ativo não pode possuir dois owners simultaneamente.

Modelar ownership de forma que o banco consiga reforçar essa invariável.

91. UNIQUE IDENTITIES

Obrigatório:

Account identity

Character name

Guild name

Item instance id

Market listing id

Trade id

Command id where persisted

92. OPTIMISTIC CONCURRENCY

Entities como:

Wallet

MarketListing

Trade

Inventory

podem utilizar:

Version

ou mecanismo equivalente.

Se houver conflito:

ConcurrencyConflict

e não overwrite silencioso.

93. DATABASE MIGRATIONS

Toda alteração de schema:

Migration

↓

Automated test

↓

Upgrade test

↓

Rollback strategy where feasible

Nunca editar migration já aplicada em produção.

Criar nova migration.

94. AUDIT EVENT

AuditEvent

```
{  
    Id  
    ActorType  
    ActorId  
    EventType  
    AggregateType  
    AggregateId  
    Payload  
    CreatedAt  
}
```

Eventos sensíveis devem ser imutáveis.

95. ADMIN AUDIT

Admin action também grava:

Before
After
Reason
AdminId

96. ANALYTICS EVENT

Não misturar necessariamente analytics com domain event.

Analytics:

```
GameplayEvent
{
    EventName
    CharacterId?
    AccountId?
    SessionId?
    Timestamp
    Properties
}
```

97. ANALYTICS PRIVACY

Não enviar informações desnecessárias.

Separar:

Operational telemetry
Gameplay analytics
Security telemetry
Personal data

98. OBSERVABILITY

OpenTelemetry pode ser usado para:

- traces;
- metrics;
- logs.

Cada request importante deve possuir correlation ID.

99. SECURITY BOUNDARY

O cliente é não confiável.

Assumir:

Client can be modified.

Client can lie.

Network can be manipulated.

Packets can be replayed.

100. CLIENT TRUST MODEL

Nunca aceitar do cliente como verdade:

Damage

Gold

Item ownership

XP

Level

Position

Cooldown

Loot

Quest completion

Craft result

101. RATE LIMITS

Aplicar por:

IP
Account
Session
Character
CommandType

Exemplo:

Chat
Trade
Market
Login
Ability
Movement

Cada um possui política diferente.

102. ABUSE DETECTION

Signals:

Impossible movement
Impossible attack rate
Impossible economy
Impossible inventory
Repeated invalid commands
Packet replay
Excessive reconnect
Bot-like timing

103. CHEAT RESPONSE

Pipeline:

Signal

↓

Score

↓

Investigation

↓

Action

Não aplicar ban permanente somente por uma heurística frágil.

104. CONTENT VALIDATOR

Antes de publicar conteúdo:

Validate IDs

Validate references

Validate ranges

Validate loot probabilities

Validate quest references

Validate item references

Validate NPC references

Validate map references

Build deve falhar se content validation falhar.

105. LOOT VALIDATION

A soma das probabilidades deve obedecer ao modelo definido.

Não permitir:

negative probability
chance > allowed range
invalid quantity
missing item

106. QUEST VALIDATION

Falhar build se:

Unknown NPC
Unknown creature
Unknown item
Unknown map
Unknown reward
Circular dependency invalid

107. ITEM VALIDATION

Falhar se:

Invalid slot
Negative weight
Negative value
Unknown requirement
Unknown ability
Invalid stack size

108. BALANCE DATA

Separar em arquivos versionados:

balance/
 combat.v1.json

```
progression.v1.json
economy.v1.json
loot.v1.json
```

109. CONTENT DATA

```
content/
  classes/
  creatures/
  items/
  abilities/
  quests/
  npcs/
  maps/
  recipes/
```

110. CONFIGURATION

```
config/
  development
  staging
  production
```

Secrets nunca entram no repository.

111. FORMULA VERSIONING

Toda fórmula relevante recebe:

```
FormulaId
Version
```

EffectiveAt

Exemplo:

DamageFormula v1

DamageFormula v2

112. BALANCE CHANGE

Mudança de balanceamento deve registrar:

OldValue

NewValue

Reason

Author

Date

Version

ExpectedImpact

113. DEATH PIPELINE

Health reaches 0

↓

CharacterDeathCommand

↓

Validate

↓

Mark Dead

↓

Resolve Death Penalty

↓

Generate Death Event

↓

Drop/Preserve Loot according to policy

↓

Schedule Respawn

114. DEATH PENALTY

MVP provisório:

- equipment durability impact;
- temporary recovery;
- possible resource loss.

XP loss:

PRODUCT_DECISION_REQUIRED

Item loss:

PRODUCT_DECISION_REQUIRED

Não implementar perdas severas até decisão explícita.

115. RESPAWN

Respawn location:

1. bound location;
2. city;
3. safe point.

Prioridade deve ser definida por configuração.

116. TELEPORT

Teleports válidos somente quando:

server-authorized

Nunca aceitar:

TeleportTo arbitrary position

do cliente.

117. WORLD INTERACTION

InteractCommand

```
{  
    TargetId  
}
```

Server validates:

- distance;
- visibility;
- interaction type;
- character state;
- target state.

118. PICKUP

PickupCommand

```
{  
    WorldItemId  
}
```

Servidor valida:

exists
nearby
not claimed
not expired
inventory capacity

119. WORLD ITEM

```
WorldItem
{
    Id
    ItemDefinitionId
    Quantity
    Position
    OwnerRestriction?
    ExpireAt?
}
```

120. LOOT OWNERSHIP

MVP:

Personal loot

ou:

Party loot

A política precisa ser definida por contexto.

PRODUCT_DECISION_REQUIRED para modelo final.

121. QUEST DISCOVERY

Quest pode ser iniciada por:

- NPC;
- item;
- localização;

- diálogo;
- evento.

Não exigir quest marker universal.

122. MAP DISCOVERY

Character mantém:

DiscoveredMapAreas
DiscoveredLocations

Pode ser usada para:

- achievements;
- rewards;
- navigation;
- secrets.

123. ACHIEVEMENT

AchievementDefinition

```
{  
    Id  
    Name  
    Criteria  
    Reward  
}
```

State:

AchievementProgress

124. NOTIFICATION

Notification

```
{  
    Id  
    CharacterId  
    Type  
    Title  
    Body  
    CreatedAt  
    ReadAt?  
}
```

Tipos:

Market
Crafting
Guild
Social
System
Event

125. OFFLINE JOB

OfflineJob

```
{  
    Id  
    CharacterId  
    Type  
    StartedAt  
    CompletesAt  
    Input  
    Status  
}
```

Não executa gameplay.

126. GAME SERVER SESSION

```
GameSession
{
    SessionId
    AccountId
    CharacterId
    ConnectionId
    ConnectedAt
    LastHeartbeatAt
    State
}
```

127. ONE CHARACTER / SESSION

MVP:

Um personagem pode possuir somente uma sessão de gameplay ativa.

Nova sessão invalida ou rejeita a anterior conforme política.

128. ACCOUNT SECURITY

Login:

```
Authenticate
↓
Issue Session
↓
Character Selection
↓
Game Session Token
```

Tokens de autenticação não devem ser reutilizados como gameplay credentials sem camada apropriada.

129. AUTHORIZATION

Toda operação server-side verifica:

Authenticated

Authorized

OwnsResource

ValidState

130. API VERSIONING

Inicial:

/api/v1

Gameplay protocol:

protocolVersion = 1

Breaking changes incrementam versão.

131. HEALTH CHECKS

Endpoints:

/health/live

/health/ready

Readiness deve verificar dependências essenciais.

132. DEPLOYMENT

Containers:

ether-api
ether-game-server
ether-worker
postgres
redis

Ambientes:

dev
staging
production

133. CI PIPELINE

Restore

↓

Build

↓

Unit Tests

↓

Architecture Tests

↓

Integration Tests

↓

Content Validation

↓

Security Checks

↓

Container Build

134. ARCHITECTURE TESTS

Garantir:

Domain does not depend on Infrastructure

Domain does not depend on API

Application does not depend on Client

Contracts contain no infrastructure dependency

135. TEST MATRIX

Domain

- damage;
- armor;
- resistance;
- XP;
- level;
- inventory;
- item ownership;
- trade;
- quest;
- death.

Application

- command handlers;
- authorization;
- transactions;
- idempotency.

Infrastructure

- repository;
- migrations;
- concurrency;
- PostgreSQL.

Game Server

- movement;
- sessions;
- replication;
- reconnect.

136. CRITICAL SECURITY TESTS

Obrigatório testar:

Client sends fake damage
Client sends fake XP
Client sends fake gold
Client sends fake item
Client sends fake position
Client repeats purchase
Client repeats trade
Client repeats loot
Client replays command
Client attacks through wall
Client attacks outside range
Client exceeds movement speed

Todos devem ser rejeitados.

137. ECONOMY TESTS

Testar:

Buy item
Sell item
Partial purchase
Concurrent purchase
Cancelled listing
Expired listing

Insufficient balance
Duplicate request
Disconnect during transaction
Server restart during transaction

138. INVENTORY TESTS

Testar:

Full inventory
Stack split
Stack merge
Equip
Unequip
Transfer
Destroy
Disconnect during transfer
Concurrent transfer
Duplicate transfer

139. COMBAT TESTS

Testar:

Basic attack
Critical
Armor
Resistance
Range
LOS
Cooldown
Mana
Death
AoE
Status effect
Disconnect

Reconnect

140. RECONNECT TESTS

Testar:

Disconnect while idle
Disconnect while moving
Disconnect during combat
Disconnect after loot
Disconnect during quest
Disconnect during market
Reconnect after 1 sec
Reconnect after 30 sec
Reconnect after session timeout

141. LOAD TEST

Cenários:

100 concurrent sessions
500 concurrent sessions
1000 concurrent sessions

Não assumir capacidade final antes de medir.

142. GAME SERVER LOAD

Medir:

Tick duration
CPU
Memory

GC
Network bandwidth
Database calls
Redis calls
Entity count
Commands/sec

143. DATABASE LOAD

Medir:

Queries/sec
Locks
Deadlocks
Connection pool
Slow queries
Index usage
Transaction duration

144. MOBILE PERFORMANCE TEST

Testar em:

Low tier
Mid tier
High tier

Durante:

- cidade;
- combate;
- muitos players;
- muitos particles;
- reconnection;
- inventory.

145. CRASH RECOVERY

Se Game Server cair:

Players reconnect

↓

Persisted state restored

Nunca assumir que memory state sobrevive.

146. WORLD RECOVERY

World state temporário:

Reconstruct from persistent state

ou:

Replay controlled events

Dependendo do subsistema.

147. BACKUP

PostgreSQL:

- backup automático;
- point-in-time recovery;
- restore tests.

Backup não é considerado válido até um restore ser testado.

148. DISASTER RECOVERY

Documentar:

RPO

RTO

antes do soft launch.

149. ECONOMIC DISASTER RECOVERY

Se exploit econômico ocorrer:

Admin precisa conseguir:

- localizar transações;
- localizar itens;
- localizar contas;
- bloquear fluxo;
- corrigir saldos;
- corrigir itens;
- auditar alterações.

150. DATA REPAIR

Nunca editar diretamente tabelas de produção sem ferramenta/auditoria.

Criar:

`AdminRepairCommand`

que gera audit.

151. STORE PURCHASE

Purchase pipeline:

```
Store
↓
Platform
↓
Receipt
↓
Backend validation
↓
Grant entitlement
↓
Audit
```

Cliente não pode declarar:

“pagamento concluído”.

152. ENTITLEMENTS

```
Entitlement
{
    AccountId
    ProductId
    Source
    ExternalTransactionId
    GrantedAt
}
```

ExternalTransactionId deve ser idempotente.

153. COSMETICS

Cosméticos não devem alterar atributos competitivos.

Outfit
MountAppearance
PetAppearance
Effect
Emote
Title

154. CONTENT OWNERSHIP

Todo conteúdo deve possuir:

ContentId
Version
Status
CreatedAt
UpdatedAt

Status:

Draft
Testing
Active
Deprecated
Archived

155. LIVE EVENT

LiveEvent
{
 Id
 Name

```
    StartAt
    EndAt
    Configuration
}
```

Não permitir overlap conflitante sem política explícita.

156. FEATURE FLAGS

Features novas devem poder ser ativadas por:

```
Environment
Platform
Region
Account cohort
Percentage
```

157. A/B TESTING

Somente depois do core estar estável.

Experimentos devem possuir:

```
ExperimentId
Variant
Cohort
Start
End
SuccessMetric
```

158. PLAYER AGENCY

O sistema deve sempre permitir que o jogador compreenda:

- por que ganhou;
- por que perdeu;
- por que não pôde executar;
- por que recebeu determinado item;
- por que uma habilidade falhou.

Evitar resultados aparentemente arbitrários.

159. FEEDBACK

Toda ação relevante deve possuir feedback:

Visual

Audio

Haptic

UI

quando apropriado.

160. CORE LOOP

Loop primário:

Explore

↓

Discover

↓

Fight

↓

Loot

↓

Improve

↓

Unlock

↓

Explore deeper

Loop secundário:

Gather



Craft



Trade



Earn



Improve

Loop social:

Meet



Party



Guild



Collaborate



Shared Goals

161. RETENTION LOOP

Curiosity



Goal



Progress



Choice



Reward



New possibility



Curiosity

Não:

Login

↓

Daily

↓

Claim

↓

Logout

162. MVP CONTENT

Primeiro mapa:

Starter Town

Áreas:

Town

Fields

Forest

Cave

Ruins

Criaturas:

Wolf

Slime

Bandit

Spider

Cave Beast

Os nomes são placeholders.

163. MVP CLASS

Apesar de três classes existirem no design:

Vertical Slice inicial deve implementar somente Warrior.

Depois:

Ranger

Arcanist

Isso reduz o custo de validar o core.

164. WARRIOR MVP

Abilities:

Basic Attack

Power Strike

Shield Guard

Dash

Somente quatro.

Objetivo:

validar combate.

165. MVP ITEMS

Categorias:

Sword

Shield

Helmet

Chest

Legs

Boots
Potion
Food
Material
QuestItem

166. MVP RARITIES

Somente:

Common
Uncommon
Rare

Epic/Legendary entram posteriormente.

167. MVP QUESTS

Criar aproximadamente:

5-10

quests de alta qualidade.

Não centenas.

168. MVP ECONOMY

Somente:

Gold
NPC shop
Basic selling

Basic buying

Marketplace completo fica para fase posterior.

169. MVP SOCIAL

Somente:

Two players

Local chat

Party

Guild posteriormente.

170. MVP DEFINITION OF FUN

O teste precisa responder:

O jogador naturalmente quer lutar contra mais uma criatura?

O jogador quer explorar a próxima área?

O loot gera curiosidade?

A progressão gera vontade de continuar?

Jogar com outra pessoa é mais interessante que jogar sozinho?

Se não:

não avançar para sistemas periféricos.

171. FIRST IMPLEMENTATION ORDER

- 001 Repository
- 002 Domain primitives
- 003 Account
- 004 Character
- 005 Authentication
- 006 Map
- 007 Position
- 008 GameSession
- 009 WebSocket
- 010 Movement
- 011 Creature
- 012 Combat
- 013 Damage
- 014 Death
- 015 Loot
- 016 Inventory
- 017 Equipment
- 018 XP
- 019 Level
- 020 Quest
- 021 Persistence
- 022 Reconnect
- 023 Two-player synchronization

172. FIRST COMMIT MILESTONE

Após os primeiros commits deve ser possível:

```
dotnet build
dotnet test
```

sem dependência do cliente.

173. FIRST GAME SERVER MILESTONE

Servidor consegue:

Start



Accept session



Authenticate



Load character



Spawn character



Receive movement



Broadcast movement

174. FIRST COMBAT MILESTONE

Servidor consegue:

Player



Target creature



Attack



Damage



Creature death



XP



Loot

175. FIRST PERSISTENCE MILESTONE

Depois:

Logout



Database



Login



Character state restored

176. FIRST RECONNECT MILESTONE

Player



Disconnect



Reconnect



Resume



No duplicated character



No duplicated loot



No lost inventory

177. FIRST SECURITY MILESTONE

Automated tests devem provar que o cliente não pode:

Teleport
Damage hack
XP hack
Gold hack
Item duplication
Cooldown bypass
Loot duplication

178. FIRST ECONOMY MILESTONE

Quando marketplace entrar:

Buyer
Seller
Listing
Transaction
Tax
Audit

devem ser testados sob concorrência.

179. RELEASE GATES

Não avançar de fase se houver:

Critical security exploit
Item duplication
Currency duplication
Data corruption
Unrecoverable reconnect
Critical crash
Broken persistence

180. IMPLEMENTATION PRINCIPLE

A equipe/agente deve implementar:

smallest valid increment

e não:

entire subsystem at once

181. AGENT TASK FORMAT

Toda tarefa futura deve seguir:

TASK

GOAL

CONTEXT

IN SCOPE

OUT OF SCOPE

BUSINESS RULES

DOMAIN IMPACT

DATA IMPACT

API IMPACT

CLIENT IMPACT

SECURITY

CONCURRENCY

TELEMETRY

TESTS

ACCEPTANCE CRITERIA

SPEC GAPS

182. AGENT MUST STOP WHEN

Encontrar:

Critical ambiguity
Conflicting business rule
Unsafe data migration
Security vulnerability
Potential duplication path
Unclear ownership

Nesse caso:

SPEC_GAP / SPEC_CONFLICT

e reportar.

183. AGENT MUST NOT

- simplificar regra crítica sem registrar;
- alterar fórmulas sem versionar;
- remover testes;
- ignorar concurrency;
- confiar no client;
- usar floating point para currency;
- criar microservices prematuramente;
- introduzir game automation;
- inventar monetização;
- adicionar sistemas fora do scope.

184. ARCHITECTURE PRINCIPLE

Começar simples.

Mas preservar:

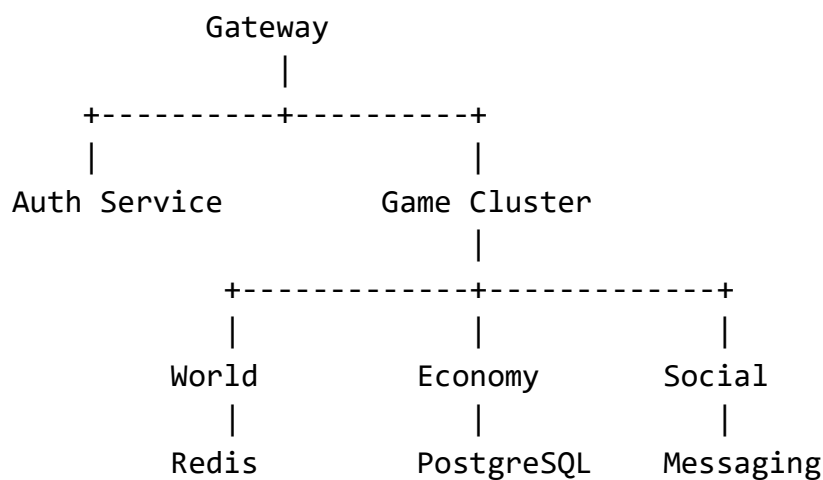
module boundaries
domain isolation
contracts
events

idempotency
observability

Assim podemos crescer sem reconstruir o projeto inteiro.

185. LONG-TERM TARGET

Arquitetura futura possível:



Mas somente quando métricas justificarem.

186. FINAL TECHNICAL RULE

Não otimizar para:

“quantos sistemas temos?”

Otimizar para:

“quantas experiências interessantes conseguimos produzir com sistemas simples, confiáveis e combináveis?”

187. FINAL PRODUCT RULE

O jogador deve sentir que:

- suas escolhas importam;
- seu personagem evolui;
- o mundo possui segredos;
- outros jogadores importam;
- seus itens possuem valor;
- seu tempo é respeitado;
- sempre existe algo interessante para descobrir.

188. DEFINITIVE FIRST TARGET

O primeiro protótipo jogável deve atingir:

2 players
1 map
1 class
5 creatures
4 abilities
10 quests
20-30 items
inventory
equipment
XP
levels
death
loot
NPC
chat
party
persistence
reconnect

Não precisamos de mais para provar o conceito.

189. O QUE NÃO SERÁ FEITO AINDA

Explicitamente fora da primeira implementação:

- Guild Wars
- Open PvP
- Housing
- Mount system
- Pet system
- Large world
- Complex crafting
- Season pass
- Premium store
- Subscription
- Advanced marketplace
- Territory control
- World bosses
- Dozens of classes
- Hundreds of quests

190. DEFINIÇÃO DO SUCESSO DO PROJETO

O sucesso inicial não é:

- 100 maps
- 1000 items
- 50 classes

É:

- Player enters
 - ↓
- Player understands
 - ↓
- Player explores
 - ↓

Player fights
↓
Player discovers
↓
Player improves
↓
Player chooses
↓
Player wants to continue

191. DECISÕES QUE DEVEM SER FECHADAS ANTES DO BETA

Final name
Lore
Art direction
Death penalty
PvP model
Final classes
Level cap
Economy values
Premium currency
Monetization
World structure
Server capacity target
Regional launch strategy

192. PRÓXIMO DOCUMENTO

Depois deste v3.0, o próximo documento não deve ser outro texto conceitual.

Deve ser:

PROJECT ETHER — v4.0

IMPLEMENTATION PACK

Contendo:

01. Complete PostgreSQL schema
02. EF Core model
03. C# domain classes
04. Aggregates
05. Value Objects
06. Commands
07. Queries
08. Domain Events
09. API contracts
10. WebSocket contracts
11. JSON schemas
12. State machines
13. Combat formulas
14. Progression tables
15. Initial content data
16. Loot tables
17. Quest definitions
18. Class definitions
19. Ability definitions
20. Docker Compose
21. CI/CD
22. Test fixtures
23. Seed data
24. Game Server loop
25. Godot client structure
26. Authentication flow
27. Reconnect flow
28. Observability
29. Security model
30. First 100 implementation tasks

Esse documento deverá ser suficientemente preciso para que um agente receba uma tarefa como:

Implement TASK ETHER-COMBAT-007.

e consiga trabalhar sem precisar perguntar:

“Mas como o dano deveria funcionar?”

porque essa resposta já estará definida no contrato.